

CoreSys Overall Review Report

Project Overview

CoreSys is a custom 64-bit UEFI-based operating system written primarily in C. The project focuses on low-level system development, including its own kernel, filesystem, boot process, drivers, and system utilities.

The project is ambitious because it attempts to build many components normally provided by mature operating systems from scratch.

1. Architecture Review

Strengths:

- Native UEFI Design: Modern boot foundation, no legacy BIOS limitations, direct UEFI services access, suitable for 64-bit systems.
- Custom Kernel Development: Full control over memory management, scheduling, security model, and experimental OS concepts.
- Modular Structure: Separation between Kernel, Drivers, Filesystem, Boot, Utilities, and Build System.

2. Programming Language Review

CoreSys uses C, which is suitable for kernel development, drivers, memory management, and UEFI interaction.

Advantages:

- Direct hardware access.
- Low overhead.
- Mature compiler support.

Challenges:

- Manual memory management.
- Pointer safety issues.
- Buffer overflow risks.

Recommendations:

- Use clang-tidy and cppcheck.
- Add kernel logging.
- Add memory poisoning.

- Add stack protection.

3. Filesystem Review

CoreSys contains a custom filesystem layer with components such as `cop_read()`, directory handling, inode management, block allocation, and file creation/removal.

Strengths:

- Custom inode system similar to Unix filesystem designs.
- Efficient metadata storage.
- Support for large files.
- 64-bit storage support.

uint64_t maximum value:

18,446,744,073,709,551,615

4. Kernel Review

CoreSys includes:

- Kernel configuration loading.
- System paths such as `/sys/kernel` and `/sys/system`.
- Device abstraction.
- Storage drivers.
- Command handling.

The kernel is moving beyond a simple hobby kernel.

5. Build System Review

CoreSys uses Makefiles.

Strengths:

- Standard for kernel development.
- Lightweight.
- Works with cross-compilers.

Observed issues:

- Missing build rules.

- Unused variables.
- Incorrect unsigned comparisons.

Recommendations:

- Use strict warnings.
- Improve dependency management.

6. Code Quality Review

Positive aspects:

- Low-level programming knowledge.
- Fixed-width integer usage.
- Hardware-oriented design.
- Custom abstractions.

Issue:

Very long identifiers reduce readability and maintainability.

Recommendation:

Use shorter structured names such as `kernel_cfg_data_t` `cfg`.

7. Security Review

Current status:

Early research / experimental OS.

Required improvements:

- Bounds checking.
- Page protection.
- Kernel/user separation.
- Privilege separation.
- Future UEFI Secure Boot support.

8. Performance Review

Advantages:

- Minimal overhead.
- No legacy components.
- Custom filesystem optimization.

Possible bottlenecks:

- Disk caching.
- Memory allocator.
- Scheduler.
- Driver architecture.

9. Comparison

Feature comparison:

CoreSys: Custom Kernel Yes, UEFI Yes, Custom Filesystem Yes, Production Ready No.

Linux: Custom Kernel Yes, UEFI Yes, Custom Filesystem Yes, Production Ready Yes.

Windows: Custom Kernel Yes, UEFI Yes, Custom Filesystem Limited, Production Ready Yes.

10. Overall Rating

Architecture: 8/10

Ambition: 10/10

Filesystem Design: 8/10

Code Organization: 7/10

Security: 5/10

Documentation: 6/10

Maintainability: 6/10

Learning Value: 10/10

Overall Score:

7.5 / 10

Final Assessment

CoreSys is a serious hobby and research operating system project, not only a bootloader experiment. The custom filesystem, kernel interfaces, UEFI foundation, and driver work

demonstrate strong understanding of operating system fundamentals.

Main improvements:

- Better documentation.
- Cleaner naming conventions.
- More modular kernel architecture.
- Memory safety protections.
- Automated testing.

With continued development, CoreSys could evolve into a complete experimental operating system platform.